Journal Pre-proof

Approximate Cutting Plane Approaches for Exact Solutions to Robust Optimization Problems

Julius Pätzold, Anita Schöbel

PII: S0377-2217(19)30971-3
DOI: https://doi.org/10.1016/j.ejor.2019.11.059
Reference: EOR 16195

To appear in: *European Journal of Operational Research*

Received date: 28 August 2018
Accepted date: 25 November 2019

Please cite this article as: Julius Pätzold, Anita Schöbel, Approximate Cutting Plane Approaches for Exact Solutions to Robust Optimization Problems, *European Journal of Operational Research* (2019), doi: https://doi.org/10.1016/j.ejor.2019.11.059

Highlights

- A modified cutting plane approach for Robust Optimization problems is proposed
- The modification consists of solving occurring subproblems only approximatively
- Convergence proofs show that this modification still finds optimal solutions
- Computational experiments show significant speed-ups gained by this modification

Approximate Cutting Plane Approaches for Exact Solutions to Robust Optimization Problems

Julius Pätzold[a,1,*], Anita Schöbel[b]

[a]*Institute for Numerical and Applied Mathematics, University of Goettingen, Lotzestraße 16 - 18, 37083 Göttingen, Germany*
[b]*Department for Mathematics, TU Kaiserslautern, Gottlieb-Daimler-Straße, 67663 Kaiserslautern and Fraunhofer Institute for Industrial Mathematics ITWM, Fraunhofer-Platz 1, 67663 Kaiserslautern, Germany*

Abstract

In this paper we deal with cutting plane approaches for robust optimization. Such approaches work iteratively by solving a robust problem with reduced uncertainty set (*robustification step*) and determining a worst-case scenario in each iteration (*pessimization step*) which is then added to the reduced uncertainty set. We propose to enhance this scheme by solving the robustification and/or the pessimization step not exactly, but only approximately, that is, until an improvement to the current solution is possible. The resulting iterative approach is called *approximate* cutting plane approach.

We prove that convergence to an optimal solution for approximate cutting plane approaches is still guaranteed under similar assumptions as for classical cutting plane approaches, in which both robustification and pessimization problem are solved exactly in each iteration. Experimentally, we investigate robust mixed integer linear optimization problems for mixed-integer polyhedral uncertainty sets of different difficulties. Solving the robustification or pessimization problem only approximately increases the number of iterations. Nevertheless, our results show that the approximate cutting plane approach becomes more efficient, in particular, if the robustification or the pessimization problem is hard.

Keywords: Robustness and Sensitivity Analysis, Robust Optimization, Mixed Integer Programming, Cutting-Plane Methods

1. Introduction

Almost every optimization problem suffers to some extent from uncertainty. In mathematics there exist different approaches to overcome this issue. In comparison to stochastic optimization, robust optimization does not require any distributional assumptions for the uncertain data. Instead, robustness concepts minimize the performance of a solution in its worst case.

*Corresponding author
Email addresses: j.paetzold@math.uni-goettingen.de (Julius Pätzold), schoebel@mathematik.uni-kl.de (Anita Schöbel)

[1]Supported by the Simulation Science Center Clausthal/Göttingen.

Robust optimization started with the work of Soyster (1973) and was extensively researched later, e.g., in Ghaoui & Lebret (1997); Ben-Tal & Nemirovski (1998, 2000); Bertsimas & Sim (2004); Ben-Tal et al. (2006), see Ben-Tal et al. (2009) for a compendium on results for strict robustness, Kouvelis & Yu (1997) for algorithms and applications for strict and regret robustness, and Goerigk & Schöbel (2016) for a survey on less conservative robustness concepts.

Solving robust counterparts of optimization problems is a challenging task due to the added nonlinearities of considering an uncertainty set. Nevertheless, there exist cases for which uncertain optimization problems can be solved exactly via reformulation (see, e.g., Ben-Tal & Nemirovski (2000); Ben-Tal et al. (2009); Bertsimas et al. (2011) and Goerigk & Schöbel (2016) for more results). On the other hand, there exist iterative (or cutting plane) approaches for tackling robust optimization problems, which are also the main focus of this paper. The standard iterative approach works in the following manner: Beginning with a small number of scenarios we determine a solution to a reduced robust problem, in which only this small uncertainty set is considered. For the resulting solution, a new scenario is determined and added to the set of scenarios. The procedure is then repeated.

This iterative approach has often been used in previous research and has been proposed and used under many different names, including Outer Approximation Method (Reemtsen (1994), Bürger et al. (2014), Goerigk & Schöbel (2016)), (modified) Benders Decomposition Approach (Montemanni (2006), Siddiqui et al. (2011)), Implementor-Adversarial Framework (Bienstock (2007)), Cutting set/plane Method (Mutapcic & Boyd (2009), Fischetti & Monaci (2012)) or Scenario Relaxation Procedure (Assavapokee et al. (2008), Aissi et al. (2009)). A proof of convergence for this iterative approach for the class of robust convex problems under strict uncertainty has been given in Mutapcic & Boyd (2009) and stems from the proof for the original cutting plane method given in Kelley (1960). The applicability of the iterative approach and its computational competitiveness holds for a wide range of optimization problems, such as robust combinatorial optimization (see Aissi et al. (2009)), robust mixed-integer programming (Fischetti & Monaci (2012) and Bertsimas et al. (2016)) and robust convex programming (Mutapcic & Boyd (2009)).

In our research, we want to generalize this iterative approach. We investigate what happens if the robustification and/or pessimization problem in each step are not solved to optimality, but only approximately. We analyze these enhancements theoretically and experimentally and show that they improve the efficiency of the resulting iterative approach significantly if the robustification and/or pessimization problem is hard to solve. Related to our proposed solution approach there already exist methods (Calafiore & Campi (2005); Campi & Garatti (2008); Calafiore (2010)) to find feasible solutions to uncertain optimization problems by constraint randomization. In contrast to our proposed approaches the main concern of this stream of literature is to investigate the probability of a solution being feasible. In contrast,

our considered uncertain optimization problem bears its uncertainty in the objective function and the set of feasible solutions is assumed to be analytically given.

The remainder of the paper is structured as follows. In Section 2 we introduce the notation, repeat the basic scheme of cutting plane approaches for robust optimization and introduce the enhanced solution scheme. Section 3 provides the theoretical analysis of convergence, while Section 4 contains the experimental results. The paper is concluded in Section 5.

2. Iterative approaches for robust optimization

As usual in uncertain optimization (see, e.g., Ben-Tal et al. (2009)) we consider a family of parameterized optimization problems $(P(u) : u \in \mathcal{U})$ where $\mathcal{U} \subseteq \mathbb{R}^q$ is a given *uncertainty set* and each scenario $u \in \mathcal{U}$ defines an optimization problem

$$P(u) \qquad \min_{x \in X} g(x, u)$$

with a set $X \subseteq \mathbb{R}^n$ and an uncertain objective function $g : X \times \mathbb{R}^q \to \mathbb{R}$. Note that we assume the function g to be defined on $\mathbb{R}^q$ and not only on the scenarios $u \in \mathcal{U}$. As common in robust optimization let us assume that one specified *nominal scenario* (the most likely one, or the undisturbed one) $u_{\text{nom}} \in \mathcal{U}$ is given.

We are interested in finding a *robust* solution to $(P(u) : u \in \mathcal{U})$. There are many different definitions when to call a solution $x \in X$ robust for $(P(u) : u \in \mathcal{U})$. In this paper we consider the most prominent definition, namely the concept of strict robustness. A solution is called *(strictly) robust* (or *minmax robust*) if it is an optimal solution to its *robust counterpart*

$$\text{(RC)} \qquad g^* = \min_{x \in X} \sup_{u \in \mathcal{U}} g(x, u),$$

see Ben-Tal et al. (2009); Soyster (1973). We consider uncertainty only in the objective and not in the constraints.

2.1. Traditional cutting plane approach for robust optimization

In the following we describe and analyze a cutting plane approach to solve (RC), i.e., to find robust solutions for a given family of optimization problems $(P(u) : u \in \mathcal{U})$. Given a subset $\mathcal{U}' \subseteq \mathcal{U}$ we define a function $g_{\mathcal{U}'} : X \to \mathbb{R}$ by

$$g_{\mathcal{U}'}(x) := \sup_{u \in \mathcal{U}'} g(x, u).$$

We can now define the following two optimization problems:

- For $\mathcal{U}' \subseteq \mathcal{U}$ the *robustification problem*

$$\text{RC}(\mathcal{U}') \qquad g^*_{\mathcal{U}'} := \min_{x \in X} \sup_{u \in \mathcal{U}'} g(x, u)$$

  is given as a relaxed robust counterpart problem considering only a subset of scenarios.

- For $x \in X$ the *pessimization problem*

$$\text{Pes}(x) \qquad\qquad g_{\mathcal{U}}(x) := \sup_{u \in \mathcal{U}} g(x,u)$$

  evaluates a solution $x \in X$ in its worst case.

Note that (RC) equals RC($\mathcal{U}$) and $g^*_{\mathcal{U}'} = \min_{x \in X} g_{\mathcal{U}'}(x)$. As we are interested in solving (RC), we define

$$g^* := g^*_{\mathcal{U}}$$

to be the objective value of an optimal solution to (RC).

In their basic version, see for example Mutapcic & Boyd (2009), iterative approaches for solving (RC) construct a sequence

$$\mathcal{U}_1 = \{u_{\text{nom}}\} \subseteq \mathcal{U}_2 \subseteq \mathcal{U}_3 \subseteq \ldots\ldots \qquad\qquad \subseteq \mathcal{U}$$

of nested scenario sets. In iteration k the algorithm finds an optimal solution x_k to the robustification problem RC($\mathcal{U}_k$). Then it determines a worst-case scenario u_k by solving the pessimization problem Pes(x_k). The scenario u_k is then added to $\mathcal{U}_k$ in order to obtain the new scenario set $\mathcal{U}_{k+1}$ for the next iteration, see Figure 1 as illustration.

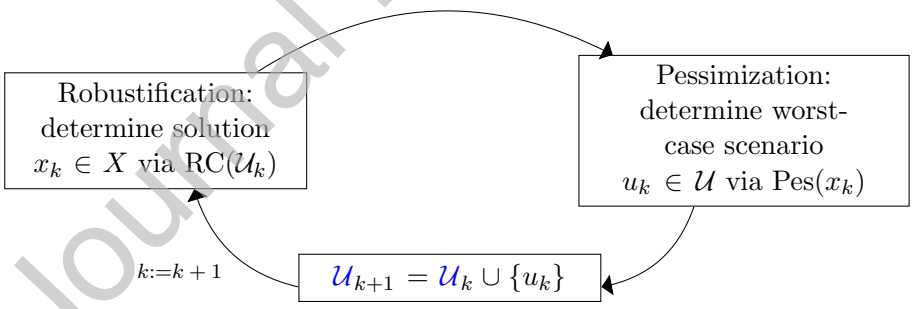

Figure 1: Basic Scheme of a cutting plane approach

We collect two elementary properties of this scheme.

Observation:

- For $\mathcal{U}_1 \subseteq \mathcal{U}_2 \subseteq \mathcal{U}$ we have $g_{\mathcal{U}_1}(x) \leq g_{\mathcal{U}_2}(x) \leq g_{\mathcal{U}}(x)$ for all $x \in X$ and hence $g^*_{\mathcal{U}_1} \leq g^*_{\mathcal{U}_2} \leq g^*_{\mathcal{U}}$.
- Let $x_{\mathcal{U}'}$ be an optimal solution to RC($\mathcal{U}'$). Then g^* is bounded by

$$g^*_{\mathcal{U}'} = g_{\mathcal{U}'}(x_{\mathcal{U}'}) \leq g^* \leq g_{\mathcal{U}}(x_{\mathcal{U}'}). \tag{1}$$

Using (1) for $\mathcal{U}' = \mathcal{U}_k$, this gives bounds on g^* in each iteration k of the cutting plane approach: $g_{\mathcal{U}}(x_k)$ is an upper bound and $g_{\mathcal{U}_k}(x_k)$ is a lower bound on g^*.

Iterative approaches usually investigate ϵ-optimality to make the algorithm stop when the result is accurate enough since they come from the idea of cutting plane algorithms.

Notation: For a given optimization problem (RC) and some $\epsilon > 0$, define a solution $x \in X$ to be $\epsilon-$optimal with respect to (RC), if $g_{\mathcal{U}}(x) - g^* \leq \epsilon$ holds.

2.2. The approximate cutting plane approach for robust optimization

Now we are able to define our enhancements to the iterative approach described in Section 2.1. The idea is to avoid solving RC($\mathcal{U}_k$) and Pes(x_k) to optimality, but save computation time by solving these problems only approximately. As we will see, this can be done without losing solution quality; i.e., the approximate cutting plane algorithm still converges to an optimal solution under similar conditions as the traditional approach of the previous section. In order to formalize this enhancement we need to define approximate versions of (RC) and (Pes).

Definition 1. *For a given problem* RC($\mathcal{U}'$) *and some threshold* $t_{opt} \in \mathbb{R}$ *we define the approximate version of* RC($\mathcal{U}'$) *to return any solution* $x \in X$ *with* $g_{\mathcal{U}'}(x) \leq t_{opt}$ *or, if no such solution exists, an optimal solution to* RC($\mathcal{U}'$).
Similarly, for a given problem Pes(x) *and some threshold* $t_{pes} \in \mathbb{R}$ *we define the approximate version of* Pes(x) *to return any scenario* $u \in \mathcal{U}$ *with* $g(x,u) \geq t_{pes}$, *or, if no such scenario exists, a worst-case scenario to* x, *i.e., an optimal solution to* Pes(x).

A-RC($\mathcal{U}', t_{opt}$) *Return some* $x \in X$ *with* $g_{\mathcal{U}'}(x) \leq t_{opt}$.
If such an x *does not exist, return* $x \in X$ *optimal to* RC($\mathcal{U}'$)

A-Pes(x, t_{pes}) *Return some* $u \in \mathcal{U}$ *with* $g(x,u) \geq t_{pes}$.
If such a u *does not exist, return* $u \in \mathcal{U}$ *optimal to* Pes(x).

This enhancement is motivated by the the following fact: If the robustification or pessimization is some mixed integer linear optimization problem, it is often very time-consuming for the MIP solver to close the last few percentage points of optimality gap. This gap-closing, however, may not be necessary since in some cases we do not need the exact solution for a certain subproblem or the exact worst-case scenario for a certain solution. Indeed, for many cases we only need some valid cut that discards the currently found solution in order to continue the iterative approach.

The choice of t_{opt} and t_{pes} determines how much computation time we need: The larger the threshold t_{opt} is in A-RC($\mathcal{U}', t_{opt}$), the more likely it is that we save computation time. For A-Pes(x, t_{pes}), the situation is reversed: The smaller the chosen threshold t_{pes}, the more computation time can be saved. Depending on t_{opt} and t_{pes} we can also force that A-RC($\mathcal{U}', t_{opt}$), or A-Pes(x, t_{pes}) are reduced to the exact problems:

Observation:

- Let lb be a lower bound on RC($\mathcal{U}'$), i.e., $lb \leq g^*_{\mathcal{U}'}$. Then for all $t_{opt} < lb$ we have that A-RC($\mathcal{U}', t_{opt}$) becomes RC($\mathcal{U}'$).
- Let ub be an upper bound on Pes(x), i.e., $ub \geq g_{\mathcal{U}}(x)$. Then for all $t_{pes} > ub$ we have that A-Pes(x, t_{pes}) becomes Pes(x).

We now formulate a version of the cutting plane algorithm, where both RC($\mathcal{U}_k$) and Pes(x_k) can be chosen to be solved approximately depending on the threshold parameters t_{opt} and t_{pes}. The algorithm is supposed to yield an ϵ-optimal solution $x_{ret} \in X$ to RC($\mathcal{U}$) and its pseudo-code is stated in Algorithm 1.

Algorithm 1: Approximate Cutting Plane Approach

Input: problem (RC), nominal scenario $u_{\text{nom}} \in \mathcal{U}$, stopping criterion $\epsilon > 0$
Output: ϵ-optimal solution $x_{ret} \in X$ to RC($\mathcal{U}$)

$\mathcal{U}_1 \leftarrow \{u_{\text{nom}}\}$, $k \leftarrow 1$, $lb_1 \leftarrow -\infty$, $ub_1 \leftarrow \infty$
while $ub_k - lb_k > \epsilon$ **do**
  determine t_{opt_k}
  $x_k \leftarrow$ solution to A-RC($\mathcal{U}_k, t_{opt_k}$)
  if A-RC($\mathcal{U}_k, t_{opt_k}$) *solved optimally and* $lb_k < g_{\mathcal{U}_k}(x_k)$ **then**
    $lb_{k+1} \leftarrow g_{\mathcal{U}_k}(x_k)$
  else
    $lb_{k+1} \leftarrow lb_k$
  determine t_{pes_k}
  $u_k \leftarrow$ solution to A-Pes(x_k, t_{pes_k})
  if A-Pes(x_k, t_{pes_k}) *solved optimally and* $ub_k > g(x_k, u_k)$ **then**
    $ub_{k+1} \leftarrow g(x_k, u_k)$
    $x_{ret} \leftarrow x_k$
  else
    $ub_{k+1} \leftarrow ub_k$
  $\mathcal{U}_{k+1} \leftarrow \mathcal{U}_k \cup \{u_k\}$
  $k \leftarrow k + 1$
end
return x_{ret}

The specific values for the threshold parameters $t_{opt_k}, t_{pes_k} \in \mathbb{R}$ are determined during the execution of the algorithm. We call the resulting sequences (t_{opt}) and (t_{pes}) *threshold sequences*. For the definition of the algorithm the threshold values could be chosen arbitrarily, but for the convergence proofs we will assume bounds on them. In the section of computational experiments choices for the threshold values will be investigated.

To ensure tractability of the algorithm, we make the following assumption.

Assumption: We assume that A-RC($\mathcal{U}_k, t_{opt_k}$) for finite $|\mathcal{U}_k|$ and A-Pes(x_k, t_{pes_k}) are solvable in a finite number of steps and their solutions are always finite.

With this assumption in mind we now investigate when Algorithm 1 converges.

3. Analysis of the approximate cutting plane approach

In this section we first give some lemmas that will be helpful to show correctness of Algorithm 1. We then prove its correctness in the case that the uncertainty set $\mathcal{U}$ is finite and then cover the case of a bounded uncertainty set $\mathcal{U}$ with possibly infinitely many scenarios. After this we analyze how the sequences t_{pes} and t_{opt} can be chosen and finally give some strategies for further speedup of the algorithm.

First, some basic properties of Algorithm 1 are collected.

Observation: For all $k \in \mathbb{N}$ it holds $lb_k \leq lb_{k+1}$, $ub_k \geq ub_{k+1}$, and $lb_k \leq g^* \leq ub_k$.

The next result shows the following: If the algorithm stops, it does so with an exact solution (up to ϵ).

Lemma 2. *If Algorithm 1 returns a solution $x_{ret} \in X$, it is an $\epsilon-$optimal solution with respect to (RC).*

Proof. Let the algorithm stop after n iterations. It hence returns a solution x_{ret} with $g_{\mathcal{U}}(x_{ret}) = ub_n$ and moreover $ub_n - lb_n \leq \epsilon$.
The lower bound lb_n at iteration n has been found in some iteration, say, $i \leq n$, i.e., $lb_n = lb_i = \min_{x\in X} g_{\mathcal{U}_i}(x)$. We get

$$ub_n - \epsilon \leq lb_n = lb_i = \min_{x\in X} g_{\mathcal{U}_i}(x) \leq \min_{x\in X} g_{\mathcal{U}}(x) = g^*$$

and hence $g_{\mathcal{U}}(x_{ret}) - g^* = ub_n - g^* \leq \epsilon$,i.e., x_{ret} is $\epsilon-$optimal. □

Next, we investigate under which conditions Algorithm 1 stops.

3.1. Convergence for finite uncertainty sets

Convergence proofs for the iterative approach in the case of a finite uncertainty set $\mathcal{U}$ are often omitted in the literature (e.g., in Aissi et al. (2009)) since it can be seen as a special case of Kelley's Cutting Plane Method (Kelley (1960)). For our proposed enhancements, however, the convergence proofs are not straightforward and we hence investigate the termination of Algorithm 1 if t_{opt} and t_{pes} are chosen dynamically during the iterations. We discuss the following different choices. Note that for Lemma 3 and 5 we assume $|X|$ to be finite because otherwise the statements are trivial.

Lemma 3. *If $t_{opt_k} < ub_k$ and $t_{pes_k} > g_{\mathcal{U}_k}(x_k)$ for all $k \in \mathbb{N}$ the number of iterations for Algorithm 1 is bounded by $|X| + |\mathcal{U}|$.*

Proof. We show that in each iteration of the algorithm either a new scenario $u_k \notin \mathcal{U}_k = \{u_1, \ldots, u_{k-1}\}$ is generated or that if the current solution x_k occurs as a solution in a later iteration of the algorithm, the algorithm will terminate. Hence, the algorithm ends after at most $|X| + |\mathcal{U}|$ iterations.

In each iteration of Algorithm 1 the problem A-Pes(x_k, t_{pes_k}) is either solved optimally or approximately.

- If A-Pes(x_k, t_{pes_k}) is solved approximately, it returns u_k with $g(x_k, u_k) \geq t_{pes_k} > g_{\mathcal{U}_k}(x_k) = \max_{u \in \mathcal{U}_k} g(x_k, u)$. Hence, $u_k \notin \mathcal{U}_k$.
- If A-Pes(x_k, t_{pes_k}) is solved optimally, then we know that the objective value $g(x_k, u_k)$ either imposes a new upper bound (line 12 of algorithm 1), or the current upper bound is already equal or better than $g(x_k, u_k)$ (line 15). In either case we get $ub_{k+1} \leq g(x_k, u_k)$.

  For every $n > k$ it hence (and because of $u_k \in \mathcal{U}_n$) holds that $g_{\mathcal{U}_n}(x_k) \geq g(x_k, u_k) \geq ub_{k+1} \geq ub_n$. We now show that for every newly obtained solution x_n (with $n > k$) the converse, i.e., $g_{\mathcal{U}_n}(x_n) < ub_n$, is true, or the algorithm terminates.

  Let $n > k$. For showing $g_{\mathcal{U}_n}(x_n) < ub_n$ we have to distinguish two cases: A-RC$(\mathcal{U}_n, t_{opt_n})$ being solved approximately and solved optimally. For an approximate solution of A-RC$(\mathcal{U}_n, t_{opt_n})$ we get $g_{\mathcal{U}_n}(x_n) \leq t_{opt_n}$ by definition of A-RC and $t_{opt_n} < ub_n$ by assumption of the lemma. For an optimal solution x_n it follows that $g_{\mathcal{U}_n}(x_n) = \min_{x \in X} g_{\mathcal{U}_n}(x) \leq \min_{x \in X} g_{\mathcal{U}}(x) \leq ub_n$. The case $g_{\mathcal{U}_n}(x_n) = ub_n$ would imply that the algorithm terminates since $lb_{n+1} \geq g_{\mathcal{U}_n}(x_n)$ (by line 6 of the algorithm) holds, which implies $ub_{n+1} \leq ub_n = lb_{n+1}$, leading to the termination of Algorithm 1. Thus for A-RC$(\mathcal{U}_n, t_{opt_n})$ being solved optimally it either holds that $g_{\mathcal{U}_n}(x_n) < ub_n$ or that the algorithm terminates.

  Therefore we have on the one hand that $g_{\mathcal{U}_n}(x_k) \geq ub_n$ and on the other hand that $g_{\mathcal{U}_n}(x_n) < ub_n$ if the algorithm does not terminate in iteration n. This means that $x_n \neq x_k$ for all $n > k$.

□

Lemma 4. *If $t_{opt_k} \leq lb_k$ and $t_{pes_k} > g_{\mathcal{U}_k}(x_k)$ for all $k \in \mathbb{N}$ the number of iterations for Algorithm 1 is bounded by $|\mathcal{U}|$.*

Proof. First note that A-RC$(\mathcal{U}_k, t_{opt_k})$ returns an optimal solution in each iteration k since $t_{opt_k} \leq lb_k$. We now show that the algorithm stops if in iteration k some $u_k \in \{u_0, \ldots, u_{k-1}\} = \mathcal{U}_k$ is generated. This bounds the number of iterations by $|\mathcal{U}|$.

Let u_n in some iteration n be the solution of problem A-Pes(x_n, t_{pes_n}). We distinguish two cases: If A-Pes(x_n, t_{pes_n}) is solved approximately, we have that $g(x_n, u_n) \geq t_{pes_n} > g_{\mathcal{U}_n}(x_n) = \max_{u \in \mathcal{U}_n} g(x_n, u)$, hence $u_n \notin \mathcal{U}_n$. Now assume that A-Pes(x_n, t_{pes_n}) is solved optimally and returns a solution u_n with $u_n \in \mathcal{U}_n$, i.e., $\max_{u \in \mathcal{U}} g(x_n, u) = \max_{u \in \mathcal{U}_n} g(x_n, u) = g(x_n, u_n)$. Then it holds that

$$lb_{n+1} \underbrace{\geq}_{\text{line 6 of Algorithm 1}} g_{\mathcal{U}_n}(x_n) = g_{\mathcal{U}}(x_n) = g(x_n, u_n) \underbrace{\geq}_{\text{line 12 of Algorithm 1}} ub_{n+1},$$

hence $ub_{n+1} - lb_{n+1} < \epsilon$ and Algorithm 1 terminates (line 2), i.e., in the case that A-Pes is solved optimally either the algorithm terminates or $u_n \notin \mathcal{U}_n$.

Overall we get in both cases that $u_k \notin \mathcal{U}_k$ or termination of the algorithm, which bounds the maximal number of iterations by $|\mathcal{U}|$. □

Lemma 5. *If $t_{opt_k} < ub_k$ and $t_{pes_k} \geq ub_k$ the number of iterations for Algorithm 1 is bounded by $|X|$.*

Proof. We show that in every iteration n of the algorithm the solution x_n has two properties: First, $g(x_n, u_n) \geq ub_{n+1}$ holds, and second, either $g_{\mathcal{U}_n}(x_n) < ub_n$ or $g_{\mathcal{U}_n}(x_n) = lb_n$ holds. In the third step, we show that $x_n = x_k$ for some $n > k$ yields either a contradiction or termination of the algorithm. This implies that the maximal number of iterations is bounded by $|X|$.

Showing the first property: The assumption $t_{pes_k} \geq ub_k$ implies that in every iteration n of the algorithm the new scenario u_n, which is found by solving A-Pes(x_n, t_{pes_n}), satisfies either $g(x_n, u_n) \geq t_{pes_n} \geq ub_n \geq ub_{n+1}$ (if A-Pes is solved approximately) or $g(x_n, u_n) \geq ub_{n+1}$ directly (if A-Pes is solved optimally). Thus $g(x_n, u_n) \geq ub_{n+1}$ holds in every iteration n.

Showing the second property: Assume that A-RC$(\mathcal{U}_n, t_{opt_n})$ finds an optimal solution x_n that hence minimizes $g_{\mathcal{U}_n}(x)$, which means that $g_{\mathcal{U}_n}(x_n) = lb_n \leq ub_n$. If A-RC$(\mathcal{U}_n, t_{opt_n})$ finds an approximate solution, it holds that $g_{\mathcal{U}_n}(x_n) \leq t_{opt_n} < ub_n$. Thus $g_{\mathcal{U}_n}(x_n) < ub_n$ or $g_{\mathcal{U}_n}(x_n) = lb_n$ holds in every iteration n.

Let now x_n be a solution returned in iteration n. If $x_n = x_k$ for some $n > k$, then we get $g(x_n, u_k) = g(x_k, u_k) \geq ub_{k+1} \geq ub_n$ implying $g_{\mathcal{U}_n}(x_n) \geq ub_n$. Now, by the second property, there exists either a contradiction to $g_{\mathcal{U}_n}(x_n) < ub_n$ from above, or termination of the algorithm since $lb_n = g_{\mathcal{U}_n}(x_n) \geq ub_n$.
Hence, $x_n \neq x_k$ for all $k < n$ and thus in every iteration a new solution x_n is found, bounding the number of iterations by $|X|$. □

From the previous two lemmas we obtain the following corollary.

Corollary 6. *If $t_{opt_k} \leq lb_k$ and $t_{pes_k} \geq ub_k$ the number of iterations for Algorithm 1 is bounded by* $\min(|X|, |\mathcal{U}|)$. *This contains the case of* A-RC *and* A-Pes *always being solved exactly.*

Proof. First, the assumptions of Lemma 5 are satisfied. This means, the number of iterations is bounded by $|X|$ and $g_{\mathcal{U}_n}(x_n) \leq ub_k$ which was shown as second property in the proof of Lemma 5. The latter guarantees that also the assumptions of Lemma 4 hold, hence the number of iterations is also bounded by $|\mathcal{U}|$. The result follows. □

Going from A-RC and A-Pes solved exactly to both solved approximately increases the maximal number of iterations from $min(|\mathcal{U}|, |X|)$ to $|\mathcal{U}| + |X|$. In the following section these bounds will not help us as both $\mathcal{U}$ and X will be assumed to possibly have infinite cardinality.

3.2. Convergence for infinite uncertainty sets

The convergence of the iterative approach for A-RC and A-Pes being solved exactly is, depending on the assumptions of X and $\mathcal{U}$, a commonly known result, see Mutapcic & Boyd (2009). The following theorem generalizes the convergence result of this paper to the case of A-RC and A-Pes being solved exactly or approximately.

Definition 7. *Given two sets $X, \mathcal{U} \subseteq \mathbb{R}$ and a function $g : X \times \mathcal{U} \to \mathbb{R}$ we define g to be uniformly Lipschitz-continuous in x with constant L if for all $u \in \mathcal{U}$ it holds*

$$|g(x, u) - g(y, u)| \leq L\|x - y\| \quad \forall x, y \in X.$$

Theorem 8. *Let X be bounded and g be uniformly Lipschitz-continuous in x with constant L. Furthermore let $t_{opt_k} \leq ub_k - \epsilon$ and $t_{pes_k} \geq ub_k$. Then Algorithm 1 converges in a finite number of steps.*

Proof. Assume Algorithm 1 does not converge. Let x_l and x_k with $l < k$ be the two solutions in iteration l and k of Algorithm 1. We know by definition of Lipschitz continuity that

$$|g(x_l, u_l) - g(x_k, u_l)| \leq L\|x_k - x_l\|$$

holds. We will show that we can bound $g(x_l, u_l) \geq ub_k$ and $g(x_k, u_l) < ub_k - \epsilon$.

Bounding $g(x_l, u_l)$: By assumption on t_{pes} it holds that $g(x_l, u_l) \geq ub_{l+1}$. This is the case because if A-Pes(x_l, t_{pes_l}) is solved approximately, then $g(x_l, u_l) \geq t_{pes_l} \geq ub_l = ub_{l+1}$ holds. If A-Pes(x_l, t_{pes_l}) is solved optimally, either $g(x_l, u_l) \geq ub_l \geq ub_{l+1}$ (line 15 of Algorithm 1), or $ub_{l+1} \leftarrow g(x_l, u_l)$ (line 12 of Algorithm 1). Overall we get $g(x_l, u_l) \geq ub_{l+1} \geq ub_k$.

Bounding $g(x_k, u_l)$: Note that $u_l \in \mathcal{U}_k$. If RC$(\mathcal{U}_k, t_{opt_k})$ is solved approximately, we get $g(x_k, u_l) \leq g_{\mathcal{U}_k}(x_k) \leq t_{opt_k} \leq ub_k - \epsilon$. If RC$(\mathcal{U}_k, t_{opt_k})$ is solved optimally we know $g(x_k, u_l) \leq g_{\mathcal{U}_k}(x_k) = lb_{k+1}$. Furthermore, $lb_{k+1} \leq ub_k - \epsilon$, otherwise the algorithm would terminate (line 2 of Algorithm 1) and we are done.

Thus we get on the one hand that $g(x_l, u_l) \geq ub_k$ and on the other hand that $g(x_k, u_l) \leq ub_k - \epsilon$. Hence,

$$\epsilon = |ub_k - (ub_k - \epsilon)| < |g(x_l, u_l) - g(x_k, u_l)| \leq L\|x_k - x_l\|.$$

With this insight we know that for all x_k with $k \in \mathbb{N}$ it holds that $\|x_k - x_l\| \geq \frac{\epsilon}{L}$ for all $l < k$. Thus every solution x_k has a minimum distance to all other solutions within X. Since X is bounded, there can only be a finite number of x_k contained in X, and Algorithm 1 has to terminate eventually. □

We remark that uniform Lipschitz continuity is a strong assumption if $\mathcal{U}$ is unbounded. In this case, even the scalar bilinear function $g : \mathbb{R} \times \mathbb{R} \to \mathbb{R}, (x, u) \mapsto x \cdot u$ can not be uniformly Lipschitz continuous. Hence, for unbounded $\mathcal{U}$ uniformly Lipschitz continuity covers only a very restrictive (i.e., not linear, not convex, not polynomial) class of functions g. Nevertheless, this case is interesting from a theoretical point of view, see Mutapcic & Boyd (2009).

In order to get rid of the boundedness assumption of X, we can assume $\mathcal{U}$ to be bounded and with a slight modification on the assumptions for the threshold sequences we will also get a convergence result.

Theorem 9. *Let $\mathcal{U}$ be bounded and g be uniformly Lipschitz-continuous in u with constant L'. Let $t_{opt_k} \leq lb_k$ and $t_{pes_k} \geq g_{\mathcal{U}_k}(x_k) + \epsilon$. Then Algorithm 1 converges in a finite number of steps.*

Proof. Assume that Algorithm 1 does not converge. Let u_l and u_k with $l < k$ be the two scenarios found in iteration l and k of Algorithm 1. We know by definition of Lipschitz continuity that

$$|g(x_k, u_k) - g(x_k, u_l)| \leq L'\|u_k - u_l\|$$

holds for all solutions $x \in X$ and especially for x_k, the solution found in iteration k. We will show that we can bound $|g(x_k, u_k) - g(x_k, u_l)| \geq \epsilon$.

Assume that A-Pes(x_k, t_{pes_k}) has been solved approximately. Hence

$$g(x_k, u_k) \geq t_{pes_k} \geq g_{\mathcal{U}_k}(x_k) + \epsilon \geq g(x_k, u_k) + \epsilon$$

and we are done. If A-Pes(x_k, t_{pes_k}) has been solved to optimality, then we know (by lines 12 and 15 of the algorithm) that $g(x_k, u_k) \geq ub_{k+1}$. For $g(x_k, u_l)$ we will look at A-RC$(\mathcal{U}_k, t_{opt_k})$. Since the threshold value t_{opt_k} is always smaller or equal lb_k, the robustification is always solved exactly and we get

$$g(x_k, u_l) \leq lb_k \leq lb_{k+1}.$$

If it would be the case that $ub_{k+1} - lb_{k+1} < \epsilon$, then the algorithm would terminate (hence after a finite number of steps). Otherwise we get

$$g(x_k, u_l) \leq lb_k \leq lb_{k+1} \leq ub_{k+1} - \epsilon.$$

Thus it follows that

$$|g(x_k, u_k) - g(x_k, u_l)| \geq |ub_{k+1} - (ub_{k+1} - \epsilon)| = \epsilon.$$

With this insight we know that for all u_k with $k \in \mathbb{N}$ it holds that $\|u_k - u_l\| \geq \frac{\epsilon}{L'}$ for all $l < k$. Thus every scenario u_k has a minimum distance to all other scenarios within $\mathcal{U}$. Since $\mathcal{U}$ is bounded, there can only be a finite number of u_k contained $\mathcal{U}$, and Algorithm 1 has to terminate eventually.

□

The next theorem allows the more general assumption on the threshold sequences, but in order to ensure convergence we now have to assume X and $\mathcal{U}$ to be bounded.

Theorem 10. *Let X and $\mathcal{U}$ be bounded and g be uniformly Lipschitz-continuous in x with constant L. Further let g be uniformly Lipschitz-continuous in u with constant L'. Finally let $t_{opt_k} \leq ub_k - \epsilon$ and $t_{pes_k} \geq g_{\mathcal{U}_k}(x_k) + \epsilon$. Then Algorithm 1 converges in a finite number of steps.*

Proof. Consider the case that the algorithm does not converge. Then either the number of A-Pes(x_k, t_{pes_k}) solved optimally or the number of A-Pes$(x_k.t_{pes_k})$ solved approximately is infinitely large. Let $I \subseteq \mathbb{N}$ denote the indices of all iterations belonging to the respective case.

Case 1, A-Pes(x_k, t_{pes_k}) solved infinitely often to optimality: For $l \in I$ we know that $g(x_l, u_l) \geq ub_{l+1}$. But for arbitrary $k \in I$ with $l < k$ we know that $g(x_k, u_l) \leq g_{\mathcal{U}_k}(x_k)$ since $u_l \in \mathcal{U}_k$. If in iteration k the problem A-RC$(\mathcal{U}_k, t_{opt_k})$ was solved to optimality, we get

$$g_{\mathcal{U}_k}(x_k) \leq lb_{k+1} \underbrace{\leq}_{\text{otherwise termination}} ub_{k+1} - \epsilon \leq ub_{l+1} - \epsilon.$$

If A-RC$(\mathcal{U}_k, t_{opt_k})$ was solved approximately we get

$$g_{\mathcal{U}_k}(x_k) \leq t_{opt_k} \leq ub_k - \epsilon \leq ub_{l+1} - \epsilon.$$

Together, we get $g(x_k, u_l) \leq g_{\mathcal{U}_k}(x_k) \leq ub_{l+1} - \epsilon$. Hence,

$$\epsilon = |ub_{l+1} - (ub_{l+1} - \epsilon)| \leq |g(x_l, u_l) - g(x_k, u_l)| \leq L\|x_l - x_k\|.$$

So all pairs x_k, x_l have a minimum distance of $\frac{\epsilon}{L}$ to each other (in X). Since we have infinitely many x_k with $k \in I$, we get a contradiction due to the boundedness of X.

Case 2, A-Pes(x_k, t_{pes_k}) solved infinitely often approximately: In this case we know that $g(x_k, u_k) \geq g_{\mathcal{U}_k}(x_k) + \epsilon$ for every x_k with $k \in I$. So for any u_l with $l \in I, l < k$ it holds that $g_{\mathcal{U}_k}(x_k) = \max_{u \in \mathcal{U}_k} g(x_k, u) \geq g(x_k, u_l)$, since $u_l \in \mathcal{U}_k$. Thus we get

$$\epsilon = |g_{\mathcal{U}_k}(x_k) + \epsilon - g_{\mathcal{U}_k}(x_k)| \leq |g(x_k, u_k) - g(x_k, u_l)| \leq L' \|u_k - u_l\|.$$

Now we know (like above with the x_k) that every pair of u_l, u_k with $k, l \in I$, $k \neq l$ has a minimum distance of $\frac{\epsilon}{L'}$ to each other. Since we have infinitely many of these, we get a contradiction to the boundedness of $\mathcal{U}$.

Thus both cases can only occur a finite number of times, hence Algorithm 1 converges. □

From these convergence results, we see that it is important to know how t_{opt} and t_{pes} can be chosen. This is further explained in the next subsection.

3.3. Choice of threshold sequences

Now that it is shown under which assumptions Algorithm 1 converges, we give a motivation on how we will choose the threshold sequences later in our experiments.

For t_{opt} it is on the one required that $t_{opt_k} < ub_k$. This indeed makes sense as otherwise we would look for a solution whose objective value might be worse than the currently best found solution. On the other hand, if t_{opt_k} is chosen to be smaller or equal lb_k, the problem is solved exactly (since there does not exist a solution with objective smaller than lb_k). This leads to the recommendation of

$$t_{opt_k} \in [lb_k, ub_k - \epsilon].$$

For t_{pes_k} it was shown that to ensure convergence of Algorithm 1 it has to hold that $t_{pes_k} \geq g_{\mathcal{U}_k}(x_k) + \epsilon$. We do not have an upper bound in $\mathbb{R}$ that guarantees to solve A-Pes(x_k, t_{pes_k}) to optimality. Nevertheless, if $t_{pes_k} \geq ub_k$ we know for the currently considered solution x_k if its objective value $g_{\mathcal{U}}(x_k)$ is worse or better than the current best solution. Hence

$$t_{pes_k} \in [g_{\mathcal{U}_k}(x_k), \infty).$$

In order to reduce the complexity of choosing the threshold values we will choose the values by

$$\begin{aligned} t_{opt_k} &= lb_k + t_{opt}(ub_k - lb_k) \\ t_{pes_k} &= ub_k + t_{pes}(g_{\mathcal{U}_k}(x_k) - ub_k), \end{aligned}$$

where we vary the parameters $t_{opt} \in [0, 1)$ and $t_{pes} \in (-\infty, 1)$.
The impact of different choices for t_{opt} and t_{pes} will be investigated in a computational study in the next chapter and we will see that a choice of $t_{pes} < 0$ does not yield better results than $t_{pes} = 0$ and hence $t_{pes} \in [0, 1)$ is a sufficient choice.

3.4. Optimization strategies

Algorithm 1 works for a wide class of problems. In our computational study we restrict ourselves to the case of X and $\mathcal{U}$ being polyhedral as well as g linear in x and u. This has the consequence that A-RC and A-Pes are mixed-integer programs. For this special case we propose three enhancements for Algorithm 1 speeding up the solution process.

Warm start for robustification and pessimization:

- For solving A-RC($\mathcal{U}_k, t_{opt_k}$), we can provide the currently best known solution x_{ret} as a starting solution.
- For solving A-Pes(x_k, t_{pes_k}) we can give $\operatorname{argmax}_{u \in \mathcal{U}_k} g(x_k, u)$ as a starting solution.

This makes in particular sense, if a MIP solver is used to solve the robustification and the pessimization problems which is the case in the setting mentioned above.

Strengthening bounds: If A-RC and A-Pes are solved approximately and a MIP solver is used, then in each iteration of Algorithm 1 we get a lower bound (in the robustification step) and an upper bound (in the pessimization step) that is found by the solver. (This is also the case when RC and Pes are solved exactly, but then we have a new solution which coincides with this bound). These bounds can also be considered and might strengthen the current bounds on the problem in some cases. Especially when the threshold values t_{pes_k} and t_{opt_k} are chosen depending on the current bounds, this improvement might affect the runtime of the algorithm.

Branch-and-Cut Framework: If A-RC is a mixed integer program, that is solved by branch-and-bound, Algorithm 1 can be implemented as a branch-and-cut algorithm. This means that we can make use of the callback function of a MIP solver in order to run the pessimization step for some obtained solution x_k and to add some scenario u_k to the set of considered solutions (corresponds to adding a new constraint to the MIP). This, on the one hand, speeds up the solution process of the robustification step since we do not have to start solving a new problem in every iteration. On the other hand this means that we have no choice but to apply the pessimization step for every feasible solution that the solver finds. This is a necessity since a MIP solver working with a branch-and-bound algorithm has to know for every node it encounters if the obtained solution can be discarded or considered feasible.

For the iterative approach with A-RC and A-Pes solved exactly the warm-start and branch-and-cut techniques have been used in the literature before, see e.g., Pérez-Galarce et al. (2014). The technique of strengthening the bound makes only sense for the approximate cutting plane approach and is hence a novelty. Nevertheless, an investigation for all three speed-up-techniques is provided in the following chapter.

4. Computational experiments

To conduct our computational studies we restrict ourselves to solve a class of robust mixed-integer linear optimization problems with mixed-integer polyhedral uncertainty. This means we consider the problem

$$\begin{aligned} P(u) \qquad \min \quad & u^T x \\ \text{s.t.} \quad & Ax \geq b, \\ & x \in \mathbb{R}^{n-p} \times \mathbb{Z}^p, \end{aligned}$$

where $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$ and for some scenario $u \in \mathcal{U} := \{u \in \mathbb{R}^{n-q} \times \mathbb{Z}^q | Cu \leq d\}$ with $C \in \mathbb{R}^{m \times n}, d \in \mathbb{R}^m$.

The convergence of Algorithm 1 when applied to this class of problems is guaranteed by Theorem 10 since $X = \{x \in \mathbb{R}^{n-p} \times \mathbb{Z}^p | Ax \geq b\}$ and $\mathcal{U}$ will be generated as compact sets and the objective function $u^T x$ is continuous and together with the compactness uniformly Lipschitz-continuous in x and u.

The robustification problem for some finite set $\mathcal{U}'$ can be formulated as a mixed integer program, i.e.,

$$\begin{aligned} \text{RC}(\mathcal{U}') \qquad \min \quad & \tau \\ \text{s.t.} \quad & Ax \geq b \\ & u^T x \leq \tau \quad \forall u \in \mathcal{U}', \\ & x \in \mathbb{R}^{n-p} \times \mathbb{Z}^p, \tau \in \mathbb{R}, \end{aligned}$$

and the pessimization step can also be modeled as a mixed integer program:

$$\begin{aligned} \text{Pes}(x) \qquad \max \quad & u^T x \\ \text{s.t.} \quad & Cu \leq d, \\ & u \in \mathbb{R}^{n-q} \times \mathbb{Z}^q. \end{aligned}$$

Generating X: We generate the coefficients for every row a_i of the matrix A to be (uniform) randomly chosen from the interval $[0, 100]$. Additionally b_i is chosen to be $10n$. Furthermore we restrict every x_i to lie in the interval $[0, 10]$ in order to ensure compactness of X.

Generating $\mathcal{U}$: The uncertainty set $\mathcal{U}$ is generated similarly. That is, every coefficient of the matrix $C \in \mathbb{R}^{m \times n}$ is chosen randomly from the interval $[0, 100]$ and all d_i are set to be $10n$. We also restrict the $u \in \mathcal{U}$ to lie in the hyper-box $[0, 10]^n$. Note that we bound the constraints $C_i u \leq d_i$ from above, whereas $A_i x \geq b_i$ bounds the x-variables from below. This is done in order to create difficult constraints, since pessimization is a maximization problem, whereas robustification is a minimization problem.

Generating easy and hard instances: For both X and $\mathcal{U}$ we make two different parameter choices: Either $p = n$ or $p = 1$, corresponding that all variables need to be integer or that only one variable needs to be integer. Hence we say that an instance has a hard robustification problem if $X \subseteq \mathbb{Z}^n$ and easy if $X \subseteq \mathbb{R}^{n-1} \times \mathbb{Z}$. Analogously, we speak of an instance with hard pessimization problem if $\mathcal{U} \subseteq \mathbb{Z}^n$ and easy if $\mathcal{U} \subseteq \mathbb{Z}^{n-1} \times \mathbb{R}$. We hence receive four different sets of instances:

- Both, robustification and pessimization are easy,
- robustification is hard (and pessimization easy),
- pessimization is hard (and robustification easy), and
- both, robustification and pessimization are hard.

As we will see, this distinction between easy and hard highly impacts the complexity of robustification and pessimization and therefore the hardness of A-RC/A-Pes.

We implemented Algorithm 1 using Python 3.6 and Gurobi 7.5.2 as IP solver (with default settings). The implementation was tested on a compute server (78GB RAM, Intel(R) Xeon(R) CPU X5650 @ 2.67GHz with four used cores). For every instance we set an overall time limit of 10 minutes.

4.1. Comparing solving exactly vs. approximately

In this first comparison we investigate if and how much replacing RC by A-RC and Pes by A-Pes speed up the iterative approach. Therefore we define four different variants of Algorithm 1.

- app-app: Both, robustification and pessimization are solved approximately with $t_{opt} = 0.5$ and $t_{pes} = 0.5$.
- ex-app: Robustification is solved exactly, pessimization approximately with $t_{opt} = 0$ and $t_{pes} = 0.5$.
- app-ex: Robustification is solved approximately, pessimization exactly with $t_{opt} = 0.5$ and $t_{pes} = -\infty$.
- ex-ex: Both, robustification and pessimization are solved exactly, i.e., $t_{opt} = 0$ and $t_{pes} = -\infty$.

Thus ex-ex corresponds to the traditional cutting plane approach of Section 2.1 and the three other variants are modifications as described in Section 2.2.

We now run all four algorithms for the four different instance sets: Both easy, robustification hard (and pessimization easy), pessimization hard (and robustification easy) and both hard.

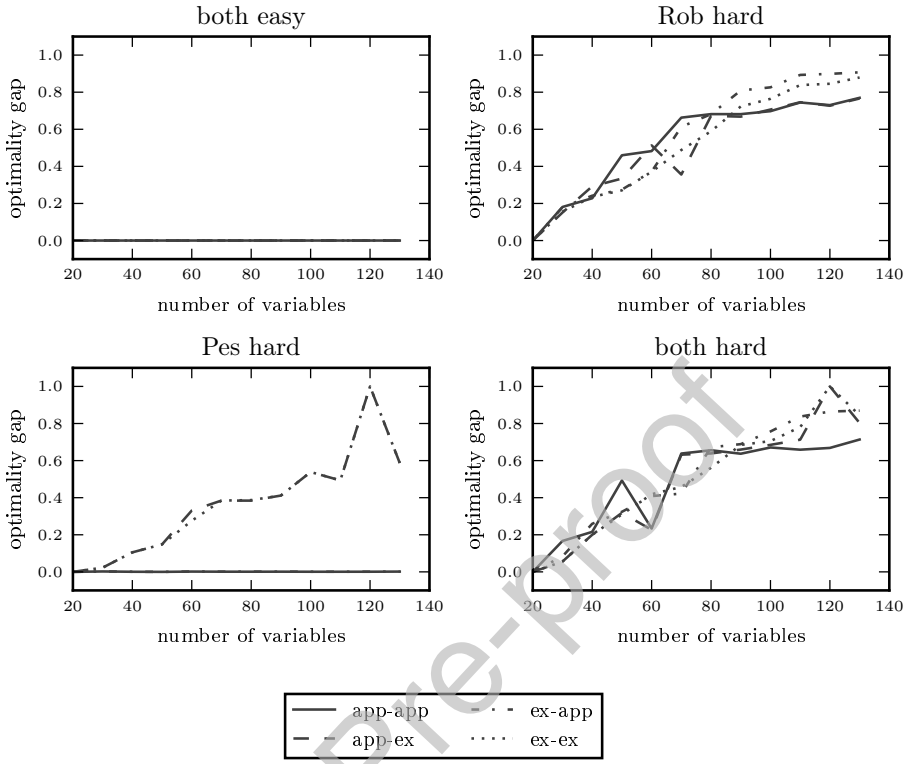

Figure 2: Comparison Exact vs. Approximately

In the first test we let the number of variables n vary between 20 and 130 and investigate the optimality gap, i.e., $\frac{ub-lb}{ub}$, obtained after hitting the time limit of 10 minutes.
For the easiest case of both robustification and pessimization easy, all algorithms find the optimal objective value within the time limit. However, when the robustification problem is hard, app-ex as well as app-app return solutions with a smaller gap than ex-app and ex-ex. If, on the other hand, the pessimization problem is difficult, the procedures app-app and ex-app have zero gap, wheres app-ex and ex-ex behave very similarly and have a large gap. If both problems are chosen to be difficult, then app-app returns the best solutions in many cases, especially if the number of variables is increasing (see Figure 2).

These results can be explained as follows: If one of robustification and pessimization is difficult and the other one is easy, solving the difficult one of them is the bottleneck. If this bottleneck problem is solved to optimality in each iteration, we might save some iterations, but we lose in terms of computation time. Therefore if the robustification problem is hard, it is preferable to use A-RC, i.e., app-ex or app-app. On the other hand, if the pessimization problem is hard it is preferable to use A-Pes, i.e., ex-app or app-app. Combining these two results suggests to use app-app if both problems are hard.

These results do not only hold for a time limit of 10 minutes, but also for the setting when

we let the algorithm solve instances to optimality. In Table 1 we show runtimes for this case.

	Both easy	RC hard	Pes hard	Both hard
App-App	0.482	44.224	379.172	14.922
App-Ex	0.48	45.01	928.298	17.278
Ex-App	0.474	100.574	379.122	21.09
Ex-Ex	0.474	98.102	929.43	21.2

Table 1: Average runtime in seconds for solving five instances with 20 variables to optimality.

Similar results are observable. If the robustification part is hard, then solving it only approximately is promising. Analogously, if the pessimization part is hard, it should be solved approximately. Overall we can say that Algorithm 1 pays off if the difficult problems are solved approximately.

4.2. Optimization strategies

We start by investigating two of the three improvements proposed in Section 3.4: Warm start and strengthening bounds. For all four different sets of instances we run four different versions of our algorithm.

- no improvements: Default version of the algorithm.
- only bounds: We update the lower bound lb_k with the lower bound found by the solver when solving A-RC and the upper bound ub_k by the upper bound from the solver when solving A-Pes.
- only warm start: When solving A-RC we provide x_{ret}, i.e., the currently best found solution, as a start solution for the robustification and when solving A-Pes we provide $\text{argmax}_{u \in \mathcal{U}_k} g(x_k, u)$, i.e., the currently worst scenario, as a start solution for the pessimization step.
- both improvements: we switch on both of the improvements.

	Both easy	RC hard	Pes hard	Both hard
both improvements	2.54	208.11	2461.19	75.41
only warm start	2.36	202.91	2500.05	74.15
only bounds	2.22	217.21	2465.08	112.8
no improvements	2.27	237.64	2658.07	106.51

Table 2: Average runtime in seconds for solving five instances with 30 variables to optimality.

We can see in Table 2 that on all instances both improvements tend to have shorter computation times, but only sometimes (for Both hard) there is a big impact on the overall computational time.

4.3. Comparison against branch-and-cut

Now we come to the third suggested improvement: The branch-and-cut framework. Based on our previous results we propose also here an approximate version of branch-and-cut in which we determine the worst-case scenario only approximately instead of solving the worst-case scenario exactly for each encountered solution as it is done in the standard branch-and-cut approaches. We hence give two versions of the branch-and-cut algorithm:

- BnC-ex is the standard case in which the worst-case scenario is determined exactly (i.e., Pes is solved for each encountered solution).
- In contrast, in the approximate version BnC-app we solve the pessimization approximately by using A-Pes with $t_{pes} = 0.5$.

We run these two algorithms on the four different classes of instances (both hard, optimization hard, pessimization hard, both easy) as in Section 4.1. For comparison we also show the solutions of app-app.

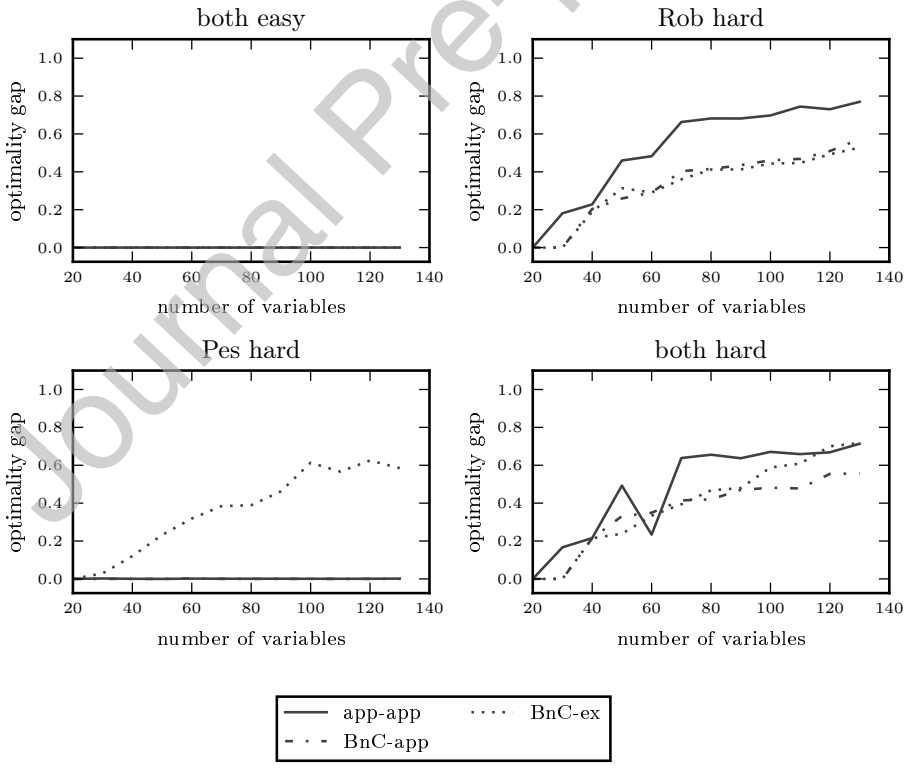

Figure 3: Comparison against branch-and-cut

It can be seen in Figure 3 that for the case of both problems being easy all algorithms return an optimal solution within the time limit. Furthermore when the robustification problem is hard the branch-and-cut algorithms seem to be a good choice. If, however, the pessimization

problem is hard, BnC-ex has a similar behaviour as ex-ex and app-ex from Section 4.1. In comparison both BnC-app and app-app find optimal solutions. When both problems are hard, we see that BnC-app yields the best results, followed by BnC-ex.

Table 3, in which we again let the algorithm solve instances to optimality, shows that app-app is the best approach if the pessimization problem is hard and the robustification is easy.

	Both easy	RC hard	Pes hard	Both hard
app-app	0.482	44.224	379.172	14.922
BnC-app	0.344	3.862	558.714	3.604
BnC-ex	0.512	5.24	1337.806	5.048

Table 3: Average runtime in seconds for solving five instances with 20 variables to optimality.

We conclude that it depends on the problem which implementation gives the best results. If the robustification problems are hard, branch-and-cut approaches seem to be superior while app-app has an advantage for instances with hard pessimization problems. This can be explained as follows: If applied to an instance with hard pessimization, the branch-and-cut approaches require too many iterations and hence have to solve too many pessimization problems, whereas app-app with a properly chosen t_{opt} does not need so many iterations and does not have to solve as many pessimization problems. This, however, might be highly dependent on the choice of t_{opt} and t_{pes}, which will be investigated in the next section.
This observation underlines a drawback of branch-and-cut: branch-and-cut can be seen as some robustification procedure with $t_{opt} = ub_k - \epsilon$, because in this branch-and-bound procedure we do not have any choice but to consider every feasible solution (and hence determine a scenario via pessimization) that has been found.

We also remark that branch-and-cut can only be applied to mixed integer linear programs while the approximate cutting plane approach app-app can be used for any kind of robust optimization problem.

4.4. Fine tuning of thresholds

Finally we investigate how different choices of t_{pes} and t_{opt} affect the solution quality. We tried different combinations for t_{opt} and t_{pes}, where a higher t_{opt} denotes, according to its definition, a higher degree of approximation ($t_{opt} = 0$ means robustification is solved exactly) and a higher t_{pes} means also a higher degree of approximation. We used instances with $n = 100$ variables and averaged over 5 instances for each problem class. In the heatmaps shown in Figure 4 the black color denotes a high optimality gap, whereas white denotes a small optimality gap.
If both problems are chosen to be easy, the problem instances could be solved for almost all parameter settings.

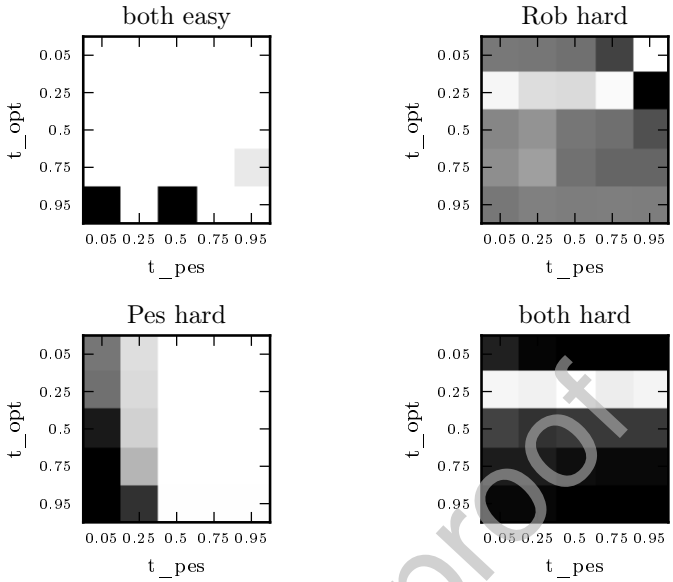

Figure 4: Different choices of the threshold parameters

If the pessimization problem is chosen to be the difficult one, we observe that the best results are obtained if the pessimization step is solved approximately with a high degree of approximation. The color gets lighter when t_{pes} increases. In this case, the results do not depend on the choice of t_{opt}. The reason is that the time spent in the robustification step is negligible as the robustification problem was chosen to be easy.

If both robustification and pessimization are hard, we see that a choice of t_{opt} of approximately 0.25 combined with a choice for t_{pes} of approximately 0.5 yields the best results. This can be explained as follows: If t_{pes} is chosen too small, we abort the pessimization step too early, hence we do not add a good cut to the problem. If, on the other hand, t_{pes} is too large, each pessimization step just takes too long. For the choice of t_{opt}, the explanation is similar. If t_{opt} is chosen too small, it takes too long to obtain the next solution. If it is chosen too large, it considers too many solutions that should be discarded.

When the robustification problem is chosen to be the difficult problem, the parameter choice behaves similarly to the case of both hard as we have seen already in the previous experiments. This could be explained by the fact that the robustification problem seems to be more difficult than the pessimization problem, which is further justified in the next section.

4.5. Summary of experiments

In general the time spent in pessimization correlates strongly with the number of iterations because finding a worst-case scenario is always the same problem. The overall time spent in robustification does not depend so much on the number of iterations. The time spent

for a single robustification namely increases with increasing iteration k since $|\mathcal{U}_k|$ increases. The pessimization on the other hand works faster because the threshold ub_k decreases with increasing k.

Overall, we have seen in this chapter that there are quite a few parameters that can be tuned in the implementation of Algorithm 1. Most importantly, we have seen that the runtime of the implemented algorithm highly depends on the choices of t_{pes} and t_{opt}. The key message to keep in mind is that a good choice highly depends on the problem type that is considered, i.e., if the pessimization or the robustification is the bottleneck. Furthermore, if the considered problems are of mixed-integer linear type, the three proposed improvements of warm start, strengthening bounds and branch-and-cut seem to be promising for speeding up the solution process.

5. Conclusion

We propose an approximate cutting plane approach for solving robust optimization problems. We introduced threshold parameters to control the degree of approximation for each subproblem (i.e., robustification and pessimization). We have shown that the approximate cutting plane approach converges to an exact optimal solution under similar conditions as traditional cutting plane approaches for robust optimization. For finite uncertainty sets and in combinatorial optimization, bounds on the number of iterations have been shown. In the computational experiments we were able to show for the broad class of robust mixed-integer linear programs that the approximate cutting plane approach outperforms the traditional one. We also propose and investigate several improvements for mixed integer linear optimization problems. The computational experiments also underline that the parameter choice should be done with precise problem knowledge in order to save significant amounts of computation time.

Further research on cutting plane approaches for robust optimization includes the following ideas. First of all, one could extend the cutting plane algorithms by dropping the assumptions we made on the solvability of A-RC($\mathcal{U}_k, t_{opt_k}$) and A-Pes(x_k, t_{pes_k}). Even if these subproblems are not bounded, cutting plane algorithms can be formulated to solve them. However, it becomes more technical to formulate a single solution algorithm like Algorithm 1 since several cases have to be considered and distinguished (e.g. a subproblem could also be unbounded or infeasible as opposed to having a finite optimal solution). Second, we only considered uncertainty in the objective function. The cutting plane approach can also be formulated for the case of uncertainty in the constraints; here one even has several possibilities on how to define a worst-case scenario, and on choosing the cuts to be added to the robustification step.

Another line of research is to extend the analysis of this paper to other robustness concepts apart from strict robustness. For example, when determining regret robustness Kouvelis &

Yu (1997) or recovery robustness (Liebchen et al. (2009); Goerigk & Schöbel (2014); Carrizosa et al. (2017)), we are faced with a hard pessimization step which makes the application of our approach in particular promising. The approach can also be applied to light robustness Schöbel (2014). It is also ongoing research to use iterative approaches for multi-objective robust problems by applying recent approaches of Botte & Schöbel (2019); Schmidt et al. (2019).

Additionally the iterative approach could be further enhanced such that robustification and pessimization run in parallel: If a solution is found, the algorithm could on the one hand go to the pessimization step for this solution. But, on the other hand, the algorithm could continue solving the robustification problem and check if a better solution can be found. If in the pessimization step a new solution has been found, we can add it as a constraint and start solving robustification again and/or start pessimization for the next solution in queue. Especially within a branch-and-cut framework this enhancement could further improve the algorithm runtime significantly.

Finally, we plan to apply the approximate cutting plane approach to real-world robust optimization problems, in particular for robust load planning (as in Bruns et al. (2014)) and for finding robust timetables (see a very recent overview in Lusby et al. (2018)). The very recent study of Pätzold (2019) deals with an adjustable robust timetabling problem that includes delay management decisions in the adjusting phase. This problem could only be solved by applying the approximate cutting plane as proposed in this paper and hence proves the applicability of our method. We also plan to apply the approximate cutting plan method in combination with other heuristics (Goerigk & Schöbel (2013)) and to apply it to other variations for robustness concepts for timetabling such as the the scenario-based approach as described in Goerigk & Schöbel (2010).

References

Aissi, H., Bazgan, C., & Vanderpooten, D. (2009). Min–max and min–max regret versions of combinatorial optimization problems: A survey. *European Journal of Operational Research*, *197*, 427–438.

Assavapokee, T., Realff, M. J., Ammons, J. C., & Hong, I.-H. (2008). Scenario relaxation algorithm for finite scenario-based min–max regret and min–max relative regret robust optimization. *Computers & operations research*, *35*, 2093–2102.

Ben-Tal, A., Ghaoui, L. E., & Nemirovski, A. (Eds.) (2006). *Special issue on Robust Optimization* volume 107:1-2 of *Mathematical Programming B*. Springer.

Ben-Tal, A., Ghaoui, L. E., & Nemirovski, A. (2009). *Robust Optimization*. Princeton and Oxford: Princeton University Press.

Ben-Tal, A., & Nemirovski, A. (1998). Robust convex optimization. *Mathematics of Operations Research*, *23*, 769–805.

Ben-Tal, A., & Nemirovski, A. (2000). Robust solutions of linear programming problems contaminated with uncertain data. *Math. Programming A*, *88*, 411–424.

Bertsimas, D., Dunning, I., & Lubin, M. (2016). Reformulation versus cutting-planes for robust optimization. *Computational Management Science*, *13*, 195–217.

Bertsimas, D., Iancu, D. A., & Parrilo, P. A. (2011). A hierarchy of near-optimal policies for multistage adaptive optimization. *IEEE Transactions on Automatic Control*, *56*, 2809–2824.

Bertsimas, D., & Sim, M. (2004). The price of robustness. *Operations Research*, *52*, 35–53.

Bienstock, D. (2007). Histogram models for robust portfolio optimization. *Journal of computational finance*, *11*, 1–64.

Botte, M., & Schöbel, A. (2019). Dominance for multi-objective robust optimization concepts. *European Journal of Operational Research*, *273*, 430–440.

Bruns, F., Goerigk, M., Knust, S., & Schöbel, A. (2014). Robust load planning of trains in intermodal transportation. *OR Spectrum*, *36*, 631–668.

Bürger, M., Notarstefano, G., & Allgöwer, F. (2014). A polyhedral approximation framework for convex and robust distributed optimization. *IEEE Transactions on Automatic Control*, *59*, 384–395.

Calafiore, G., & Campi, M. C. (2005). Uncertain convex programs: randomized solutions and confidence levels. *Mathematical Programming*, *102*, 25–46.

Calafiore, G. C. (2010). Random convex programs. *SIAM Journal on Optimization*, *20*, 3427–3464.

Campi, M. C., & Garatti, S. (2008). The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, *19*, 1211–1230.

Carrizosa, E., Goerigk, M., & Schöbel, A. (2017). A biobjective approach to recovery robustness based on location planning. *European Journal of Operational Research*, *261*, 421–435.

Fischetti, M., & Monaci, M. (2012). Cutting plane versus compact formulations for uncertain (integer) linear programs. *Mathematical Programming Computation*, *4*, 239–273.

Ghaoui, L. E., & Lebret, H. (1997). Robust solutions to least-squares problems with uncertain data. *SIAM Journal of Matrix Anal. Appl.*, *18*, 1035–1064.

Goerigk, M., & Schöbel, A. (2010). An empirical analysis of robustness concepts for timetabling. In T. Erlebach, & M. Lübbecke (Eds.), *Proceedings of ATMOS10* (pp. 100–113). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik volume 14 of *OpenAccess Series in Informatics (OASIcs)*. URL: http://drops.dagstuhl.de/opus/volltexte/2010/2753. doi:http://dx.doi.org/10.4230/OASIcs.ATMOS.2010.100.

Goerigk, M., & Schöbel, A. (2013). Improving the modulo simplex algorithm for large-scale periodic timetabling. *Computers and Operations Research*, *40*, 1363–1370.

Goerigk, M., & Schöbel, A. (2014). Recovery-to-optimality: A new two-stage approach to robustness with an application to aperiodic timetabling. *Computers and Operations Research*, *52*, 1–15.

Goerigk, M., & Schöbel, A. (2016). Algorithm engineering in robust optimization. In L. Kliemann, & P. Sanders (Eds.), *Algorithm Engineering: Selected Results and Surveys* (pp. 245–279). volume 9220 of *LNCS State of the Art*. URL: http://arxiv.org/abs/1505.04901.

Kelley, J. E., Jr (1960). The cutting-plane method for solving convex programs. *Journal of the society for Industrial and Applied Mathematics*, *8*, 703–712.

Kouvelis, P., & Yu, G. (1997). *Robust Discrete Optimization and Its Applications*. Kluwer Academic Publishers.

Liebchen, C., Lübbecke, M., Möhring, R. H., & Stiller, S. (2009). The concept of recoverable robustness, linear programming recovery, and railway applications. In R. K. Ahuja, R. Möhring, & C. Zaroliagis (Eds.), *Robust and online large-scale optimization*. Springer volume 5868 of *Lecture Note on Computer Science*.

Lusby, R., Larsen, J., & Bull, S. (2018). A survey ob robustness in railway planning. *European Journal of Operationsaal Research*, *266*, 1–15.

Montemanni, R. (2006). A benders decomposition approach for the robust spanning tree problem with interval data. *European Journal of Operational Research*, *174*, 1479–1490.

Mutapcic, A., & Boyd, S. (2009). Cutting-set methods for robust convex optimization with pessimizing oracles. *Optimization Methods & Software*, *24*, 381–406.

Pätzold, J. (2019). Finding Robust Periodic Timetables by Integrating Delay Management. Submitted.

Pérez-Galarce, F., Álvarez-Miranda, E., Candia-Véjar, A., & Toth, P. (2014). On exact solutions for the minmax regret spanning tree problem. *Computers & Operations Research*, *47*, 114–122.

Reemtsen, R. (1994). Some outer approximation methods for semi-infinite optimization problems. *Journal of Computational and Applied Mathematics*, *53*, 87–108.

Schmidt, M., Schöbel, A., & Thom, L. (2019). Min-ordering and max-ordering scalarization methods for multi-objective robust optimization. *European Journal of Operational Research*, *275*, 446–459.

Schöbel, A. (2014). Generalized light robustness and the trade-off between robustness and nominal quality. *MMOR*, *80*, 161–191.

Siddiqui, S., Azarm, S., & Gabriel, S. (2011). A modified benders decomposition method for efficient robust optimization under interval uncertainty. *Structural and Multidisciplinary Optimization*, *44*, 259–275.

Soyster, A. (1973). Convex programming with set-inclusive constraints and applications to inexact linear programming. *Operations Research*, *21*, 1154–1157.